

ME403 — Lab 1: Warming up to the Robot!

Sabanci University

Please complete the following tasks on a robot of your choice (Dobot Magician or MG400).

Assignment

Task 1. Derive the formulation of the end-effector position in the robot's frame by using homogeneous transformation matrices of the links. Then, please write a function that utilizes these matrices to estimate the tip position. All the required dimensions such as link lengths and relative axis locations can be obtained by simply measuring them at one of the robots, or by referring to the robot manual, which can be found online. This part of your code should estimate where the robot's tip would end up in the Cartesian space when it is at a certain configuration (i.e. for certain joint angles).

Task 2. Instruct the robot to move to 20 random different configurations within its workspace. At each configuration collect the position feedback provided by the robot and store that data in an array. Compare those feedback values against your code's estimation. If your forward kinematics model is alright, your estimation and the values returned from the robot should have a very good match. If you see significant errors, please review your kinematic model. Show the agreement between measured and estimated position values by plotting the magnitude of the difference between them.

Let $e = p - p_m$, where $p = [x \ y \ z]^T$ is the estimated position and $p_m = [x_m \ y_m \ z_m]^T$ is the position provided by the robot. Please present the magnitude of the error in those 20 configuration samples with a box plot.

Task 3. Write a function that obtains the Jacobian of the end-effector and outputs the matrix to the terminal window. This function should get the current configuration as input and provide the matrix at will.

Due Date & Deliverables

Due Date: As announced by the instructor.

Deliverables: Please complete these steps on your own by the time of your lab session. A brief report will be requested after the lab session. An assignment for that post-lab document will be announced later.

Robot Kinematics Reference

Dobot Magician

The Magician is a 4-DOF parallel-arm robot. The firmware reports joint angles in an absolute-reference convention (not relative between links):

Joint	Zero posture	Range
J1	base aligned with $+X$	$\pm 90^\circ$
J2	upper arm vertical ($+Z$)	0° to 85°
J3	forearm horizontal ($+X$), absolute world angle	-10° to 85°
J4	wrist yaw (no FK role)	$\pm 90^\circ$

Geometric FK ($L_1 = 135$ mm, $L_2 = 147$ mm):

$$R = L_1 \sin J_2 + L_2 \cos J_3, \quad z = L_1 \cos J_2 - L_2 \sin J_3, \quad x = R \cos J_1, \quad y = R \sin J_1$$

At $(J_1, J_2, J_3) = (0, 0, 0)$: TCP = (147, 0, 135) mm (bare flange).

End-Effector TCP Offset (Magician)

The firmware adds an (o_x, o_y, o_z) bias to every `GetPose` result. For FK labs use the bare-flange preset so measurements match the formula above directly.

Preset	TCP offset (mm)	Home at (0,0,0,0)
none (bare flange)	(0, 0, 0)	(147, 0, 135) mm
motor	(60, 0, 0)	(207, 0, 135) mm
suction	(60, 0, -70)	(207, 0, 65) mm — physical cup tip

Note: DobotStudio's "suction cup" factory preset = (+60,0,0) (motor-shaft TCP). Use `motor` preset to match DobotStudio `GetPose`. Use `suction` to model the physical cup tip in FK/IK calculations.

Set the offset from Python:

```
import utils as U
U.set_tool_offset(bot, *U.MAGICIAN_TOOL_OFFSETS["none"]) # bare
flange
ox, oy, oz = U.get_tool_offset(bot) # verify
print(ox, oy, oz) # should print 0.0 0.0 0.0
```

DOBOT MG400

The MG400 is a 4-DOF parallel-arm robot (440 mm reach). Its firmware reports J_3 as an absolute angle ($J_3^{\text{fw}} = J_2 + J_3^{\text{body}}$); `utils.py` handles the conversion automatically.

Joint	Zero posture	Range
J1	base aligned with $+X$	$\pm 160^\circ$
J2	shoulder link horizontal	-25° to $+85^\circ$
J3	firmware absolute ($= J_2 + J_3^{\text{body}}$)	-25° to $+105^\circ$
J4	wrist rotation	$\pm 180^\circ$

Geometric FK ($Z_{\text{base}} = 116$ mm, $L_1 = L_2 = 175$ mm):

$$R = L_1 \cos J_2 + L_2 \cos(J_3^{\text{fw}} - J_2), \quad z = Z_{\text{base}} + L_1 \sin J_2 + L_2 \sin(J_3^{\text{fw}} - J_2)$$

$$x = R \cos J_1, \quad y = R \sin J_1$$

MG400 labs use the bare TCP (no EE attachment); no offset switching needed. Safe Z range: 5 to 140 mm (Z cannot go negative).

Provided Files

utils.py	Robot connection and motion helpers.
interface.py	Terminal interface to jog the robot and run your code.
myCode.py	Write your code here (inside run()).
requirements.txt	Python dependencies.

Quick Start — If You Attended Lab 0

Your virtual environment is already set up. Re-activate it and install the requirements. **Do this at the start of every lab**, as we may update the packages between sessions.

Linux / macOS

```
# From Students/01_SecondWeek/  
source .venv/bin/activate  
pip install -r requirements.txt
```

Windows (PowerShell)

```
# From Students\01_SecondWeek\  
.\.venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

Running in Simulation (Recommended First Step)

Simulation Quick Start

Run in simulation before using real hardware. No USB/Ethernet connection needed.

```
# Default: Magician simulation (MuJoCo)  
python interface.py  
  
# MG400 simulation  
DOBOT_ROBOT_TYPE=mg400 python interface.py  
  
# PyBullet backend (if MuJoCo unavailable)  
DOBOT_SIM_BACKEND=pybullet python interface.py  
  
# Headless (no GUI window)  
DOBOT_VIZ=0 python interface.py
```

Useful environment variables:

Variable	Effect
DOBOT_SIMULATION=1	Use simulation (default)
DOBOT_SIMULATION=0	Use real robot
DOBOT_ROBOT_TYPE=magician mg400	Select robot
DOBOT_SIM_BACKEND=mujoco pybullet	Select sim backend
DOBOT_EE=none motor suction	Magician TCP mode (default: none)
DOBOT_VIZ=0	Disable GUI windows

Select Your Robot

Open `utils.py` and set the robot type at the top before running anything:

```
ROBOT_TYPE = "magician"      # or "mg400"
MG400_IP   = "192.168.2.x"  # change to your assigned robot's IP (MG400
                             only)
```

If you are using the Dobot Magician

Check the “**Working with Dobot Magician**” file on Sucourse before working with the Magician.

How to Run

```
python interface.py
```

At the prompt:

- `move q1 q2 q3 q4` — jog the robot to the given joint angles
- `x` — execute your `run()` function in `myCode.py`
- `q` — quit and close the connection

First-Time Setup — If You Did Not Attend Lab 0

Note: Avoid PDF Copy-Paste for Commands

Type commands manually or copy from the digital version. Some PDF viewers insert extra spaces in folder paths (e.g. `. venv` instead of `.venv`).

1 — Virtual Environment

Linux / macOS

```
# From Students/01_SecondWeek/  
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

Windows (PowerShell)

```
# From Students\01_SecondWeek\  
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
pip install -r requirements.txt
```

2 — Dobot Magician (USB/Serial)

The Magician connects over USB via a CP210x Silicon Labs chip at 115 200 baud. `pydobotplus` handles the serial protocol; the firmware runs on the robot's internal controller. Close Dobot-Studio before running any Python script — only one process can hold the serial port at a time.

Linux only — grant port access once per machine, then log out and back in:

```
sudo usermod -a -G dialout $USER  
ls /dev/ttyUSB*      # verify: should show /dev/ttyUSB0
```

Windows — the CP210x driver installs automatically in most cases. If not, download it from Silicon Labs. The port shows up as COM3 (or similar) in Device Manager under *Ports (COM & LPT)*.

3 — DOBOT MG400 (Ethernet/TCP-IP)

The MG400 does not use serial; it communicates over Ethernet via three TCP ports:

Port 29999	Dashboard	Enable/disable, error query, pose and angle readout
Port 30003	Move API	MovJ, MovL, Arc, RelMovJ, Sync, ...
Port 30004	Feedback	1440-byte packets every 8 ms (live robot state)

Network — set your PC's Ethernet adapter to static IP 192.168.2.100, netmask 255.255.255.0, then ping your assigned robot:

```
ping 192.168.2.7      # Robot 1 -- replace with your assigned robot's IP
```

Robot IP addresses:

```
Robot 1  192.168.2.7
Robot 2  192.168.2.10
Robot 3  192.168.2.9
Robot 4  192.168.2.6
```

SDK — the MG400 requires Dobot's Python SDK, which is not on PyPI. Clone it once from the `dobot_ws/` root:

```
git clone https://github.com/Dobot-Arm/TCP-IP-4Axis-Python.git \
  vendor/TCP-IP-4Axis-Python
```

`utils_mg400.py` picks up `dobot_api.py` from `vendor/` automatically — no further configuration needed.